Explore Image Filtering Solution

Copyright Barry Van Veen 2016

Contents

- Prepare Workspace
- Problem 1a
- Problem 1b
- Problem 1c
- Problem 2a
- Problem 2b
- Problem 2c
- Problem 3a
- Problem 3b

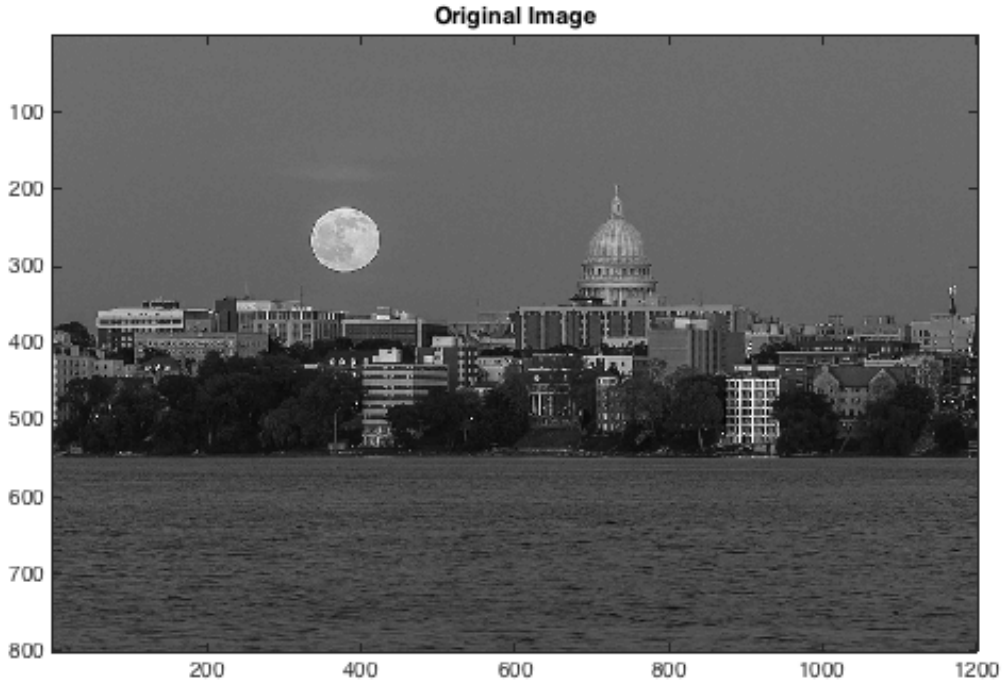
Prepare Workspace

```
clear
close all
clc
load Moon.mat % loads image named 'MadisonMoon'

im = double(MadisonMoon); % convert from uint8 to double precision format

% Display image

figure
imagesc(im)
axis equal
axis tight
colormap(gray)
title('Original Image')
```

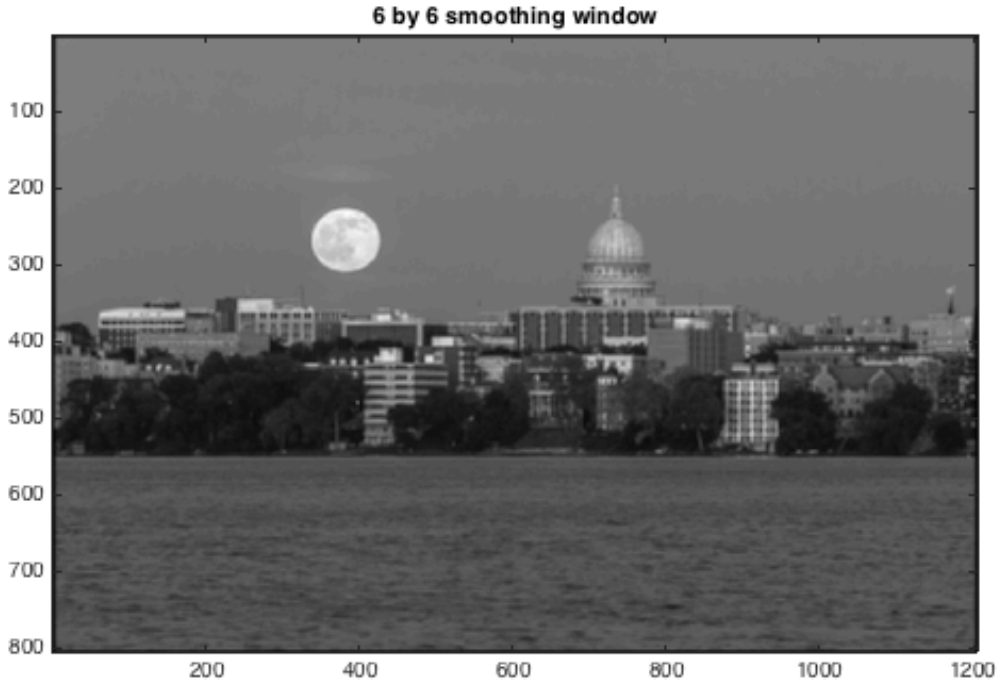


Problem 1a

For this part we are averaging 6 by 6 neighborhoods of pixels. This will blur the image.

```
h_a = ones(6,6)/36;
ima = conv2(im,h_a);

figure
imagesc(ima)
axis equal
axis tight
colormap(gray)
title('6 by 6 smoothing window')
```

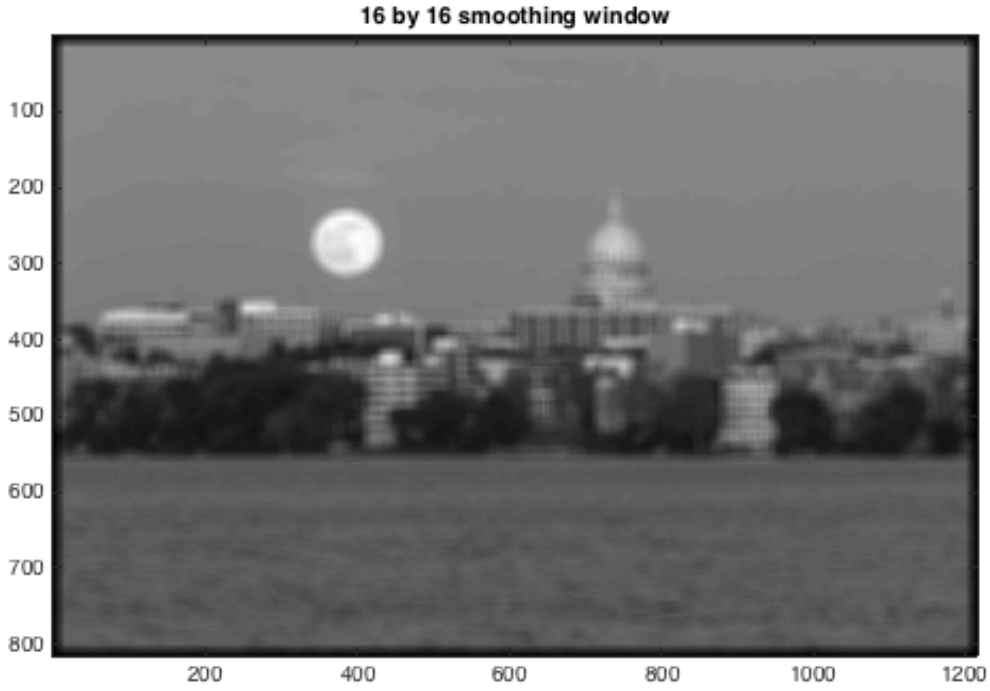


Problem 1b

The larger the neighborhood of pixels that we average over the more blurred the result becomes. The result of using a 16 by 16 neighborhood to average over will result in this image being more blurry than part a.

```
hb = ones(16,16)/256;
imb = conv2(im,hb);

figure
imagesc(imb)
axis equal
axis tight
colormap(gray)
title('16 by 16 smoothing window')
```



Problem 1c

The two images will be the same. If I do a conv2 of hh and hv I will get the point spread function ha from part a. The overall result of doing both vertical and horizontal blurrings in sequence will be the same as if I did the blurring from part a.

```
hh = ones(1,6)/6;
hv = hh';

im_h = conv2(im,hh);
im_hv = conv2(im_h,hv);

figure
imagesc(im_hv)
axis equal
axis tight
colormap(gray)
title('Horiz then Vertically Blurred Image')

norm(ima-im_hv,'fro')

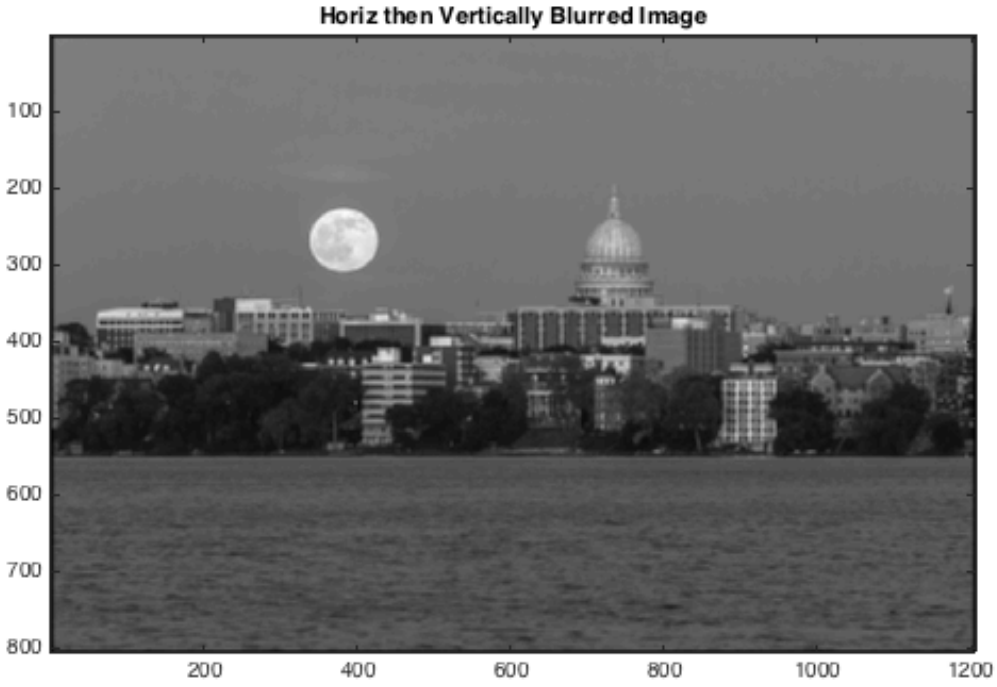
figure
image(ima-im_hv)
axis equal
axis tight
colormap(gray)
title('Difference of horiz/vert-blurred image and part a')

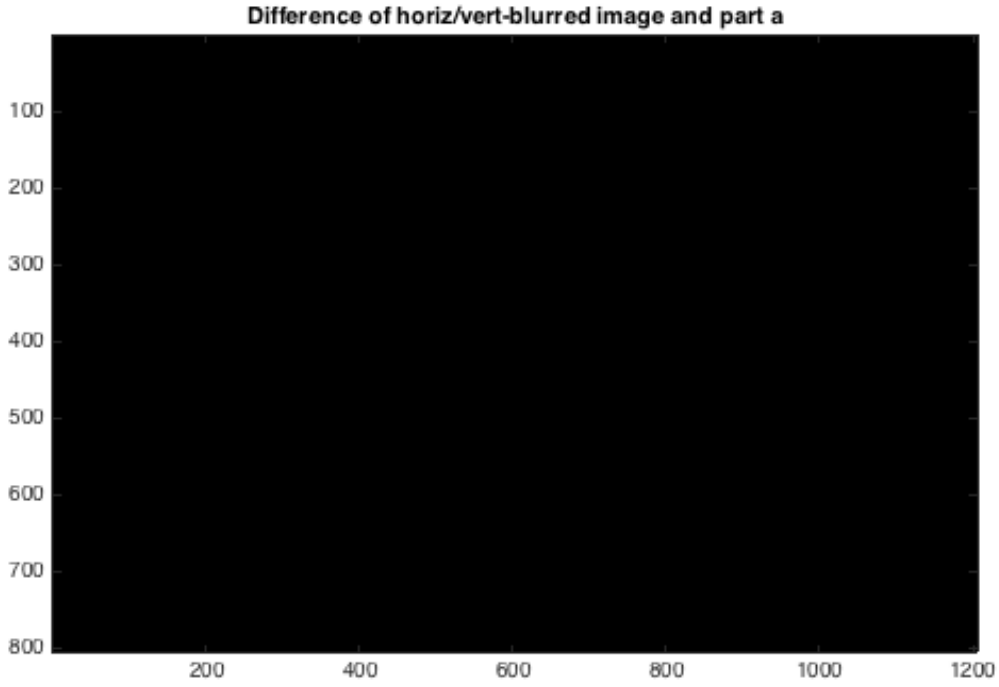
% Note that the image command was used instead of imagesc.  Imagesc is
% deceptive in this case since the differences between imb and imcc are on
```

```
% the order of 10^{-14}.
```

```
ans =

    1.7632e-11
```



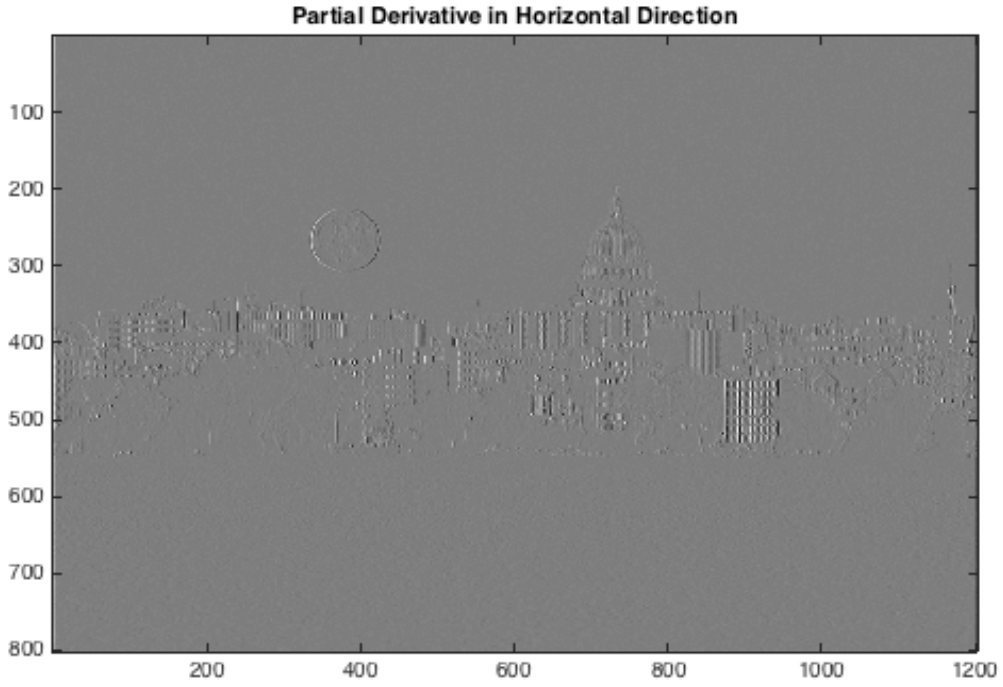


Problem 2a

```
hdh = [1 0 -1;2 0 -2;1 0 -1];

imhoriz = conv2(im,hdh);

figure
imagesc(imhoriz)
axis equal
axis tight
colormap(gray)
title('Partial Derivative in Horizontal Direction')
```

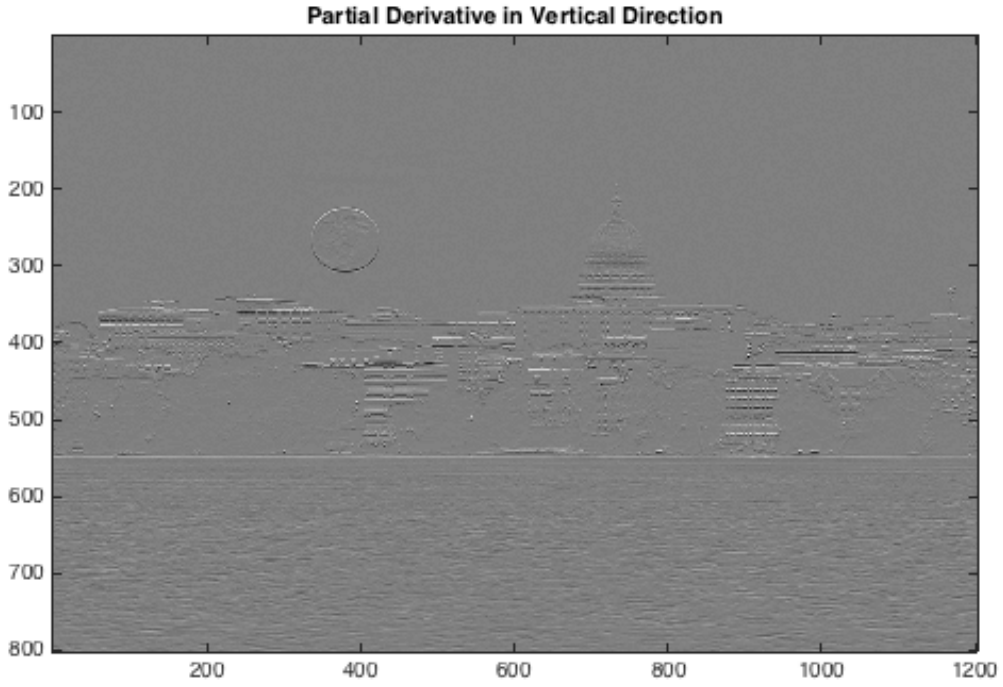


Problem 2b

```
hdv = [1 2 1;0 0 0;-1 -2 -1];

imvert = conv2(im,hdv);

figure
imagesc(imvert)
axis equal
axis tight
colormap(gray)
title('Partial Derivative in Vertical Direction')
```



Problem 2c

White pixels represent regions where the original image transitions from dark to light.

Black pixels are negative values and represent regions where the original image transitions from light to dark.

Grey pixels represent regions where the image is approximately constant, that is, the values are not changing.

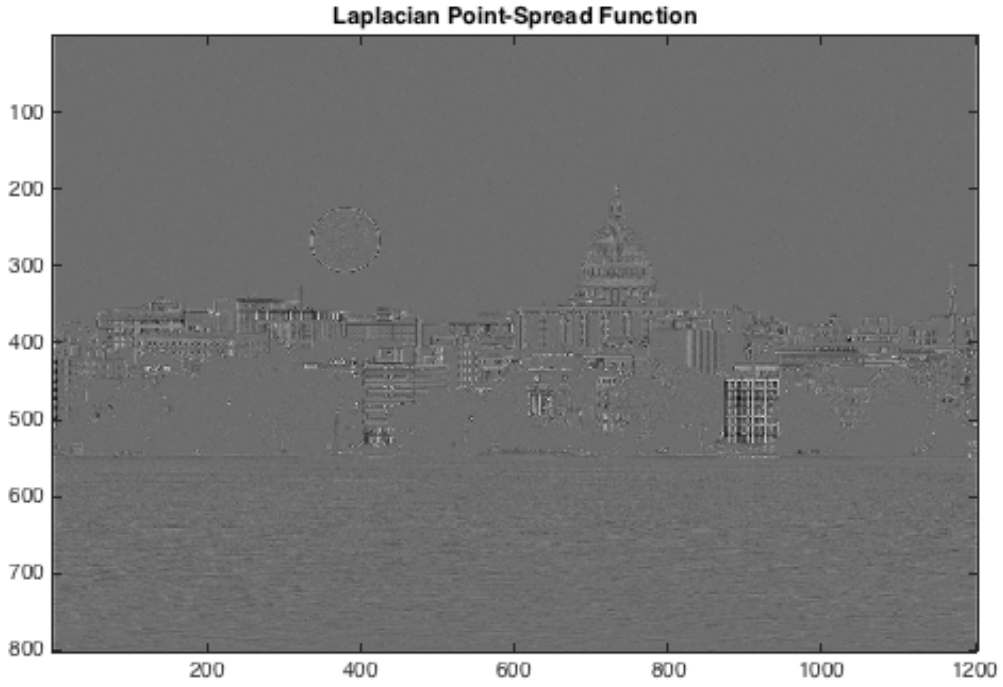
imhoriz is based on differences in the vertical direction, so it will tend to emphasize horizontal edges in the image. Hence the surface of the water still shows texture, and we see the roofs and other horizontal edges in the buildings. Differences in nearby pixels are greatest near edges.

imvert is based on differences in the horizontal direction, so it will tend to emphasize vertical edges in the image. Hence the surface of the water now looks constant, and we see the sides and other vertical edges in the buildings.

Problem 3a

```
L = [-1 -1 -1; -1 8 -1; -1 -1 -1];

imL = conv2(im,L);
figure
imagesc(imL)
axis equal
axis tight
colormap(gray)
title('Laplacian Point-Spread Function')
```



Problem 3b

The Laplacian picks out both vertical and horizontal edges in the image, and thus would be good for detecting edges of objects.

Published with MATLAB® R2015a